

Forecast Verification (ngen-verf)

Table of Contents

- [Introduction](#)
 - [1. Fetch NWM Streamflow Forecast Data](#)
 - [2. Fetch Streamflow Observation Data](#)
 - [3. Pair Observations with Forecast Data](#)
 - [4. Compute Evaluation Metrics](#)
 - [5. Generate Visualization of Metrics](#)
- [Installation & Usage](#)
 - [Run ngen-verf from docker container](#)
 - [1. Log into docker with your credentials](#)
 - [2. Pull the latest ngen-verf docker image](#)
 - [3. Start an ngen-verf container](#)
 - [4. Set up configuration for verification](#)
 - [5. Run the verification application](#)
 - [6. Repeat steps 4 & 5 to run another verification application](#)
 - [Clone, build, and run ngen-verf locally](#)
- [Configure verification applications](#)
 - [Sample config file](#)
 - [Explanation for section 1: general](#)
 - [Explanation for section 2: file_paths](#)
 - [Explanation for section 3: nwm_forecast](#)
 - [Explanation for section 4: flow_observation](#)
 - [Explanation for section 5: pair_data](#)
 - [Explanation for section 6: metrics](#)
 - [Explanation for section 7: plots](#)
- [Verification datasets and outputs](#)
 - [Sample screen output](#)
 - [Directory structure](#)
 - [Notes on the Directory Structure](#)
 - [Notes on NWM Forecast Data](#)
 - [Sample plots](#)
 - [Sample spatial maps](#)
 - [Sample boxplots](#)
 - [Sample histograms](#)
- [Metrics Supported](#)

Introduction

ngen-verf is a command-line tool designed for verifying NWM streamflow forecasts across short-range, medium-range, and long-range configurations (although currently it only support short-range). It can also be used to compare verification statistics between different forecast datasets, whether from different NWM versions or forecast periods.

The **ngen-verf** workflow consists of five sequential steps:

1. Fetch NWM Streamflow Forecast Data

In this step, NWM streamflow forecast data are downloaded based on the specified configuration, versions, and time periods defined in the YAML configuration file. The data are then extracted for the designated locations. **ngen-verf** leverages the Python library **TEEHR** ([Tools for Explanatory Evaluation in Hydrologic Research](#)) to retrieve operational NWM forecasts from Google Cloud Storage (GCS) and store them as Parquet files in the TEEHR data model.

This step can be highly resource-intensive and time-consuming.

2. Fetch Streamflow Observation Data

In this step, streamflow observations are downloaded for the specified time periods and locations as defined in the YAML configuration file. Currently, **ngen-verf** supports only **USGS streamflow observations**.

3. Pair Observations with Forecast Data

Next, streamflow forecasts are paired with corresponding observations based on validation time and location. This step requires both forecast and observation datasets to be fully downloaded and processed in the preceding steps.

4. Compute Evaluation Metrics

Once forecasts and observations are paired, **ngen-verf** calculates performance evaluation statistics for the lead times and metrics defined in the YAML configuration file. These computations are based on the paired datasets from the previous step.

5. Generate Visualization of Metrics

In the final step, visualizations are created for the lead times and metrics specified in the YAML configuration file. **ngen-verf** produces three types of plots:

- **Spatial maps** – displaying geographic patterns in forecast performance
- **Histograms** – illustrating the distribution of metric values
- **Boxplots** – summarizing metric variability across different conditions

Installation & Usage

Note: This section is written for RTX users only (to run ngen-verf in AWP workspace). OWP users should refer to the UAT document for how to run ngen-verf.

Run ngen-verf from docker container

1. Log into docker with your credentials

```
docker login registry.sh.nextgenwaterprediction.com
```

2. Pull the latest ngen-verf docker image

```
docker pull registry.sh.nextgenwaterprediction.com/ngwpc/nwm-ngen/ngen-verf:latest && docker tag registry.sh.nextgenwaterprediction.com/ngwpc/nwm-ngen/ngen-verf:latest ngen-verf
```

3. Start an ngen-verf container

```
docker run -v [DIR_LOCAL]:[DIR_CONTAINER] -it --entrypoint /bin/bash ngen-verf
```

#The following example command will launch an ngen-verf container, mapping my home directory to the same directory within the container, so that I can always access all data stored in my home directory in the same way, regardless whether I run verification locally or from a container.

```
docker run -v /home/youqiong.liu:/home/youqiong.liu -it --entrypoint /bin/bash ngen-verf
```

4. Set up configuration for verification

Make a copy of the example configuration file ([/ngen-app/ngen-verf/sample_files/config.yaml](#)) and adjust the contents as need to set up your configuration. Sample gage files needed for the [file_paths](#) section of the configuration file can be found in [/ngen-app/ngen-verf/sample_files](#).

5. Run the verification application

```
python /ngen-app/ngen-verf/verification.py config.yaml
```

6. Repeat steps 4 & 5 to run another verification application

Clone, build, and run ngen-verf locally

Clone ngen-verf from Gitlab

```
cd [NGEN_VERF_ROOT]
git clone -b development --recurse-submodules https://gitlab.sh.nextgenwaterprediction.com/NGWPC/nwm-ngen/ngen-verf.git
```

Clone ngen-eval from Gitlab

ngen-verf requires ngen-eval as a dependency

```
cd [NGEN_EVAL_ROOT]
git clone -b development --recurse-submodules https://gitlab.sh.nextgenwaterprediction.com/NGWPC/nwm-ngen/ngen-eval.git
```

Create python venv

```
cd [VENV_ROOT]
/usr/bin/python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

Install ngen-eval

```
cd [NGEN_EVAL_ROOT]/ngen-eval
pip install .
```

Install ngen-verf

```
cd [NGEN_VERF_ROOT]/ngen-verf
pip install .
```

where [NGEN_EVAL_ROOT], [NGEN_VERF_ROOT], [VENV_ROOT] refer to the directory to install ngen-eval, ngen-verf, and python venv in your local workspace, respectively.

Run ngen-verf locally

1) Set up your verification application via a configuration yaml file (e.g., config.yaml)

Follow the sample config file (ngen-verf/sample_files/config.yaml) and the comments therein to set up the configurations for your verification application as needed.

2) run the verification script with the configuration

```
python [NGEN_VERF_ROOT]/ngen-verf/verification.py config.yaml
```

3) Repeat the first two steps as many times as needed

Configure verification applications

Sample config file

```
#### configuration for the verification run
general:

    # define which of the 5 steps to run; each step can be run independently,
    # assuming data from previous steps (from previous runs) are available for use
    steps:
        fetch_fcst_data: true
        fetch_obs_data: true
        pair_data: true
        compute_metrics: true
        plot_metrics: true

    # user-specified name for the set of locations to carry out verification for
    # this name is used to create a folder in 'data_dir_root' (in 'file_paths' section) to store all the datasets
    # and outputs
    # see function 'data_paths' in ngen-verf/src/ngen/verf/settings.py to see how the overall data directory
    # structure is defined
    location_set_name: calib_basin_group1

    # [optional] list of usgs gage IDs or NWM links IDs; if blank, a location_list_file must be provided in
    # 'file_paths' section
    # if both location_list and location_list_file are provided, location_list takes precedence
    location_list:

    # [optional] must be specified if location_list is not blank; currently supported options: usgs_gage,
    # nwm30_link
    location_type:

    variable_name: streamflow # currently only 'streamflow' is supported
    nwm_configuration: short_range # Currently, only short_range, short_range_hawaii, and
    short_range_puertorico are supported

    # user-specified name of datasets (used for creating sub-folders for datasets and plots)
    # must be a list of one or more strings
    dataset_name: [v3_sep, v3_oct]

    # list of NWM versions; must have the same length as 'dataset_name'. Currently only option is "nwm30"
    nwm_version: [nwm30, nwm30]

    # list of start/end dates for forecast verification period; list must have the same length as 'dataset_name'
    # date ranges specified must be applicable to the corresponding NWM archive in Google Cloud
```

```

forecast_start_date: ['2024-09-01', '2024-10-01']
forecast_end_date: ['2024-09-30', '2024-10-30']

#### configurations for various file paths
# sample files can be obtained from repo at ngen-verf/sample_files/gage_files
file_paths:

# root directory to store the downloaded NWM forecast and flow observation data
data_dir_root: /home/yuqiong.liu/work/data/ngen-verf

# [optional] but either location_list (in 'general' section) or location_list_file must be specified
# tab or comma delimited csv file, and must contain either a "gage" or
# a "link" column specifying the locations to conduct verification for, with NWM version indicated in column
name (e.g., nwm30_link)
location_list_file: /home/yuqiong.liu/work/Gitlab/ngen-verf/sample_files/gage_files
/usgs_gages_link_CONUS_calib100.csv # a sample of 100 calibrated locations over conus
#location_list_file: usgs_gage_link_all_CONUS.tsv #all gages for CONUS domain

# parquet files containing the crosswalk between NWM feature_id and usgs gage id
# note a crosswalk file should be provided for each NWM version listed in 'general/nwm_version'
# see utils/create_crosswalk.py on how the sample crosswalk parquet was created
# sample file usgs_nwm30_crosswalk_all_domains.parquet includes 8866 gages
crosswalk_file:
nwm30:
    /home/yuqiong.liu/work/Gitlab/ngen-verf/sample_files/gage_files/usgs_nwm30_crosswalk_all_domains.parquet
#nwm40:

# meta data for USGS gages (sample file gages_metadata_all_domains.csv includes 8660 USGS gages)
gage_meta_file: /home/yuqiong.liu/work/Gitlab/ngen-verf/sample_files/gage_files/gages_metadata_all_domains.csv

# geometry (point lat/lon) for USGS gages; see utils/create_geometry.py on how the sample geometry parquet
was created
# sample file usgs_point_geometry_CONUS.parquet includes 8186 USGS gages
geometry_file: /home/yuqiong.liu/work/Gitlab/ngen-verf/sample_files/gage_files
/usgs_point_geometry_all_domains.parquet

#### configurations for retrieving NWM forecast data (typically no need to change)
nwm_forecast:
    fetch_fcst: [true, true] # whether to fetch the forecast data for each dataset listed in ['general']
['dataset_name']
    output_type: channel_rt
    t_minus: [0, 1, 2] # Only used if an assimilation run is selected
    data_source: GCS # Specifies the remote location from which to fetch the data ("GCS", "NOMADS", "DSTOR")
    kerchunk_method: local # When data_source = "GCS", specifies the preference in creating Kerchunk reference
json files.
# "local" - always create new json files from netcdf files in GCS and save locally,
if they do not already exist
# "remote" - read the CIROH pre-generated jsons from s3, ignoring any that are
unavailable
# "auto" - read the CIROH pre-generated jsons from s3, and create any that are
unavailable, storing locally

    process_by_z_hour: true # If True, NWM files will be processed by z-hour per day. If False, files will be
# processed in chunks (defined by STEPSIZE). This can help if you want to read many
reaches
# at once (all ~2.7 million for medium range for example).

    stepsize: 100 # Only used if PROCESS_BY_Z_HOUR = False. Controls how many files are processed in memory at
once
# Higher values can increase performance at the expense on memory (default value: 100)

    ignore_missing_file: false # If True, the missing file(s) will be skipped and the process will resume
# If False, TEEHR will fail if a missing NWM file is encountered

    overwrite_output: false # If True, existing output files will be overwritten; if False, existing files are
retained

    memory_per_worker_gb: 3 # configurable memory (in GB) assigned to each worker

#### configurations for retrieving streamflow observation data (currently only usgs is supported)
#### typically no need to change selections in this section

```

```

flow_observation:
  usgs: # flow obs retrieved will be stored at [data_dir_root]/[location_set_name]/usgs
  chunk_by: day # data are saved in parquet files by day/month/year
  overwrite_output: false # If True, existing output files will be overwritten; if False, existing files are
retained
  memory_per_worker_gb: 3 # configurable memory (in GB) assigned to each worker

#### paired data generation
pair_data:

  # whether to overwrite existing joined parquet files
  overwrite: true

  # break large location list into smaller groups (for pair_data and cacL_metrics steps) to avoid potential
memory issues and to speed up processing
  group_size: 200

#### configuration for metrics calculation
metrics:

  # whether to overwrite existing metric files
  overwrite: true

  # library to be used for metric calculation; currently supported options: teeHR, ngen.eval
  # see the lists of supported metrics toward the end of sample config.yaml file (also ngen-verf/src/ngen/verf
/settings.py)
  library: ngen.eval

  # list of metrics to be calculated
  # if set to 'all', all available metrics will be calculated
  # [caution] some metrics available in ngen.eval (e.g., event-based metrics) may be time consuming to compute
#metric_subset: [KGE, NSE, CORR, NNSE]
metric_subset: 'all'

  # list of metrics to be excluded from calculation (leave blank if no need to exclude any metrics)
  # here the event-based metrics are excluded because they are time consuming to run
metric_exclude: [PKBIAS, PKTE, EVBIAS, FBIAS]

  # non-exceedance probability threshold of streamflow to be used for categorical metrics in ngen.eval
flow_threshold_categorical: 0.9

  # non-exceedance probability threshold of streamflow to be used for event-based metrics in ngen.eval
flow_threshold_event: 0.9

  # list of lead times (in hours) for which metrics should be calculated for
  # the lead times can be integers (e.g., 1,2), or an interval (e.g., 1-3), or 'all'
  # if set to 'all', all raw lead times (i.e., 1, 2, ..., 18 in the case of short-range forecasts) will be
calculated
#lead_times: [1,3,5,'1-3','1-5','4-6','6-10']
lead_times: [all, 1-5, 6-10, 11-18]

#### configuration for plots
# For each type of plots (histogram, boxplot, spatial map),
# 1) 'metric_subset' must be a subset of metrics defined for calculation (i.e., 'metric_subset' defined in
Section 'metrics')
# 2) 'lead_times' must be a subset of lead times defined for metrics calculation (i.e., 'lead_times' defined
in Section 'metrics')
plots:
  histogram:

    # whether to create histogram plots
    plot: true

    # list of metrics to create histogram plots for
    metric_subset: [KGE, NSE, CORR, NNSE]

    # for each metric, define list of bin edges for generating the histogram plot
    # For 'KGE','NSE','CORR' and 'NNSE', default bins (see settings.py) will be used if not defined.
    # For other metrics, the data will be cut into 8 equal-width bins if bin edges are not provided
    binning:
      KGE: [-inf, -1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0]

```

```

NSE: [-inf, -1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0]
NNSE: [0, 0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 1.0]
CORR: [-1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0]

# list of lead times (in hours) to create histograms for
lead_times: [1, 3, 5, 10, 15, 18, 1-5, 6-10, 11-18]

# define a string that gets added to the filename of histogram plots to differentiate between plots
# of different lead time groups or different binning scenarios, to avoid overwriting the existing plots.
# Leave it blank if overwriting is desired
tag:

boxplot: # the same comments for histogram apply here (except for binning)
plot: true
metric_subset: [KGE, NSE, CORR, NNSE]

# Define an appropriate range (min/max) for each metric to scale the boxplot.
# Default ranges (see settings.py) will be applied for 'KGE', 'NSE', 'CORR', and 'NNSE' if not specified.
# For other metrics, if a range is not defined, values will remain unscaled,
# which may result in improperly displayed statistics due to outliers.
scaling:
KGE: [-0.5, 1]
NSE: [-0.5, 1]
CORR: [-0.5, 1]
NNSE: [0, 1]
RSR: [0, 20]
MAE: [0, 5]
RMSE: [0, 20]
PBIAS: [-100, 300]

lead_times: [1, 3, 5, 10, 15, 18, 1-5, 6-10, 10-18]
show_outliers: false
tag:

spatial_map:
plot: true

# list of metrics to create spatial maps for
metric_subset: [KGE, NSE, CORR, NNSE]

# Define an appropriate range (min/max) for each metric to scale the values for the spatial map
scaling:
KGE: [-0.5, 1]
NSE: [-0.5, 1]
CORR: [-0.5, 1]
NNSE: [0, 1]
RSR: [0, 20]
MAE: [0, 20]
RMSE: [0, 20]
PBIAS: [-100, 300]

lead_times: [1, 3, 5, 10, 15, 18, 1-5, 6-10, 11-18]

# define a string that gets added to the filename of spatial map plots to avoid overwriting the existing
ones (if any)
tag:

##### DONE WITH CONFIGURATIONS #####
#----- List of currently supported metrics for teeHR and ngen.eval -----
# both short and long names of the metrics are accepted
#
### supported metrics for teeHR
# 'n_obs': 'primary_count',
# 'n_mod': 'secondary_count',
# 'min_obs': 'primary_minimum',
# 'min_mod': 'secondary_minimum',
# 'max_obs': 'primary_maximum',
# 'max_mod': 'secondary_maximum',
# 'mean_obs': 'primary_average',
# 'mean_mod': 'secondary_average',
# 'sum_obs': 'primary_sum',

```

```
# 'sum_mod': 'secondary_sum',
# 'var_obs': 'primary_variance',
# 'var_mod': 'secondary_variance',
# 'max_delta': 'max_value_delta',
# 'NSE': 'nash_sutcliffe_efficiency',
# 'NNSE': 'nash_sutcliffe_efficiency_normalized',
# 'KGE': 'kling_gupta_efficiency',
# 'KGE1': 'kling_gupta_efficiency_mod1',
# 'KGE2': 'kling_gupta_efficiency_mod2',
# 'ME': 'mean_error',
# 'MAE': 'mean_absolute_error',
# 'MSE': 'mean_squared_error',
# 'RMSE': 'root_mean_squared_error',
# 'pt_obs': 'primary_max_value_time',
# 'pt_mod': 'secondary_max_value_time',
# 'pt_err': 'max_value_timedelta',
# 'RBIAS': 'relative_bias',
# 'MBAIS': 'multiplicative_bias',
# 'RMAE': 'mean_absolute_relative_error',
# 'CORR': 'pearson_correlation',
# 'sCORR': 'spearman_correlation',
# 'R2': 'r_squared',
# 'aprBIAS': 'annual_peak_relative_bias',
```

```
##### supported metrics for ngen.eval
# 'CORR': 'pearson correlation',
# 'NSE': 'nash_sutcliffe_efficiency',
# 'NNSE': 'nash_sutcliffe_efficiency_normalized',
# 'NSElog': "logarithmic nash_sutcliffe_efficiency",
# 'NSEwt': 'weighted NSE and NSElog',
# 'KGE': 'kling_gupta_efficiency',
# 'MAE': 'mean_absolute_error',
# 'RMSE': 'root_mean_squared_error',
# 'PBIAS': 'percent_bias',
# 'RSR': 'RMSE_observation_std_ratio',
# 'HSEG_FDC': 'pbias_high_flow_FDC',
# 'MSEG_FDC': 'pbias_medium_flow_FDC',
# 'LSEG_FDC': 'pbias_low_flow_FDC',
# 'POD': 'probability_of_detection',
# 'FAR': 'false_alarm_ratio',
# 'CSI': 'critical_success_index',
# 'FBIAS': 'frequency_bias',
# 'PKBIAS': 'percent_peak_flow_bias',
# 'PKTE': 'peak_timing_error',
# 'EVBIAS': 'event_volume_bias',
```

Explanation for section 1: **general**

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
steps	fetch_fcst_data	No	Boolean, True or False	True	Whether to fetch NWM forecasts in the current run; setting to False to skip the step
	fetch_obs_data	No	Boolean, True or False	True	Whether to fetch USGS streamflow observations in the current run; setting to False to skip the step. Note this step can be run independent from the first step: fetch_fcst_data
	pair_data	No	Boolean, True or False	True	Whether to join the time series of NWM forecasts and observation data based on location and valid time in the current run Note: this step depends on data fetched in the previous two steps: fetch_fcst_data , fetch_obs_data
	compute_metrics	No	Boolean, True or False	True	Whether to compute metrics in the current run. Note this step depends on paired datasets generated from the previous step: pair_data

	plot_metrics	No	Boolean, True or False	True	Whether to create plots of metrics in the current run. Note this step depends on the metric results calculated in the previous step: compute_metrics
location_set_name	No	String	calib_basin_group1		User-specified name for the set of locations to carry out verification for. This string is used to create a folder in data_dir_root (in file_paths section) to store all the datasets and outputs. See the Verification datasets and outputs section for details the overall data directory structure
location_list	Yes	List of strings or integers	[01123000, 01010000] or [8134650, 23963741]		List of usgs gage IDs (strings) or NWM links IDs (integers), with length of list ≥ 1 . Note for list of strings, it is optional to put quotation marks around each string, i.e., both [01123000, 01010000] and ['01123000', '01010000'] would work. Either location_list or location_list_file (in file_paths section) must be specified. If both are defined, location_list will take priority
location_type	Yes	String	usgs_gage, or nwm30_link,		Must be specified if location_list is not blank; currently supported options: usgs_gage, nwm30_link, indicating whether the location IDs specified in location_list is USGS gages, or NWM v30 feature (or link) IDs.
variable_name	No	String	streamflow		Currently only option is streamflow
nwm_configuration	No	String	short_range		String indicating the NWM configuration to conduct verification for. Currently only <i>short_range</i> , <i>short_range_hawaii</i> , and <i>short_range_puertorico</i> are supported. TEEHR has an issue in processing <i>short_range_alaska</i> forecasts; hence that configuration is currently not supported.
dataset_name	No	List of strings	['v3_sep', 'v3_oct']		User-specified names for the datasets to conduct verification for and to be compared with each other. These strings will be used in the names of the data directories or files or plots. See the Verification datasets and outputs section . The number of datasets can be any integer ≥ 1
nwm_version	No	List of strings	['nwm30', 'nwm30']		List of strings indicating NWM versions; list must have the same length as dataset_name . Currently accepts "nwm30"
forecast_start_date	No	List of datetime strings	['2024-09-01', '2024-10-01']		List of start dates for forecast verification period; List must have the same length as dataset_name
forecast_end_date	No	List of datetime strings	['2024-09-30', '2024-10-30']		List of end dates for forecast verification period; List must have the same length as dataset_name

Explanation for section 2: **file_paths**

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
data_dir_root		No	String	/home/yuqiong.liu /work/data/ngen-verf	Root directory to store the downloaded NWM forecast and flow observation data, as well as the paired data, the calculated metrics, and the plots generated. See the Verification dataset and output section for details the overall data directory structure. If this root directory does not already exist, it will be created.
location_list_file		Yes	FilePath	/home/yuqiong.liu /work/data/ngen-verf /usgs_gages_link_CO NUS_calib100.csv	Path to a tab or comma delimited csv file, which must contain either a "gage" or "link" column specifying the locations to conduct verification for, with NWM version indicated in column name (e.g., nwm30_link) Either location_list (in general section) or location_list_file must be specified. If both are defined, location_list takes priority
crosswalk_file	nwm30	Yes	FilePath	/home/yuqiong.liu /work/Gitlab/ngen-verf /sample_files /gage_files /usgs_nwm30_crosswalk_all_domains.parquet	Only needed for NWM versions defined in nwm_version in the general section. The sample file includes 8866 gages all four NWM domains.
	nwm40	Yes	FilePath		Only needed for NWM versions defined in nwm_version in the general section
gage_meta_file		No	FilePath	/home/yuqiong.liu /work/Gitlab/ngen-verf /sample_files /gage_files /gages_metadata_all_domains.csv	Path to a tab or comma delimited csv file containing some meta data information (e.g., agency, name, NWS ID etc) for each gage. Currently, this file is only used to determine whether a given gage ID is a USGS gage, requiring minimally a column "gage" and another column "agency". See <code>utils/create_crosswalk.py</code> on how the sample crosswalk parquet was created. The sample file includes 8660 USGS gages across all four NWM domains.
geometry_file		No	FilePath	/home/yuqiong.liu /work/Gitlab/ngen-verf /sample_files /gage_files /usgs_point_geometry_all_domains.parquet	Path to a parquet file containing the geometry (point lat/lon) for USGS gages; see <code>utils/create_geometry.py</code> on how the sample geometry parquet was created. The sample file includes 8186 USGS gages across all four NWM domains.

Explanation for section 3: **nwm_forecast**

This section is used to set up configurations for retrieving and processing NWM forecast data using the TEEHR library. Typically there is no need to change entries in this section except for the first one.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
fetch_fcst		No	List of Boolean	[True, True]	Whether to fetch the forecast data for each dataset listed in dataset_name in the general section. List must have the same length as dataset_name
output_type		No	String	channel_rt	Currently only option is 'channel_rt' (i.e., the streamflow simulation from NWM forecasts)
t_minus		No	List of integers	[0, 1, 2]	Only used if an assimilation run is selected
data_source		No	String	GCS	Specifies the remote location from which to fetch the data ("GCS", "NOMADS", "DSTOR")
kerchunk_method		No	String	local	When data_source = "GCS", specifies the preference in creating Kerchunk reference json files. "local" - always create new json files from netcdf files in GCS and save locally, if they do not already exist "remote" - read the CIROH pre-generated jsons from s3, ignoring any that are unavailable "auto" - read the CIROH pre-generated jsons from s3, and create any that are unavailable, storing locally
process_by_z_hour		No	Boolean	True	If True, NWM files will be processed by z-hour per day. If False, files will be processed in chunks (defined by STEPSIZE). This can help if you want to read many reaches at once (all ~2.7 million for medium range for example).
stepsize		No	Integer	100	Only used if PROCESS_BY_Z_HOUR = False. Controls how many files are processed in memory at once. Higher values can increase performance at the expense on memory (default value: 100)
ignore_missing_file		No	Boolean	True	If True, the missing file(s) will be skipped and the process will resume. If False, TEEHR will fail if a missing NWM file is encountered
overwrite_output		No	Boolean	False	If True, existing output files will be overwritten; if False, existing files are retained. It is recommended to set it to False to allow using existing parquet files that are previously downloaded and processed.
memory_per_worker_gb		Yes	Integer	3	Configurable memory (in GB) assigned to each worker. Default = 3 (GB), which is typically sufficient for all verification jobs. If needed, increase this number to accommodate extremely large location sets.

Explanation for section 4: **flow_observation**

This section is used to set up configurations for downloading streamflow observation data using the TEEHR library. Currently, only USGS streamflow data is supported. Typically there is no need to change entries in this section.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
usgs	chunk_by	No	String	day	Data are saved in parquet files by day
	overwrite_output	No	Boolean	False	If True, existing output files will be overwritten; if False, existing files are retained
	memory_per_worker_gb	Yes	Integer	3	Configurable memory (in GB) assigned to each worker. Default = 3 (GB), which is typically sufficient for all verification jobs. If needed, increase this number to accommodate extremely large location sets.

Explanation for section 5: **pair_data**

Currently only one field is needed for this section.

Field	Optional?	Value Type	Sample Value	Notes
overwrite	No	Boolean	False	If True, existing paired files will be overwritten; if False, existing files are retained
group_size	Yes	Integer	200	Number of locations in each smaller group to break the location list into, to avoid potential memory issues (and speed up processing)

Explanation for section 6: **metrics**

The section is used to set up configurations for metrics calculation.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
overwrite		No	Boolean	True	whether to overwrite existing metric files
library		No	String	ngen.eval	Python library to be used for metric calculation; currently supported options: <i>teehr</i> , <i>ngen.eval</i> . The list of supported metrics for each library can be found at the Metrics supported section. Note <i>ngen.eval</i> is built on the metric functions used by the calibration software ngen-cal.
metric_subset		Yes	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to be calculated. If set to 'all', all available metrics supported by the chosen library will be calculated Caution: some metrics available in ngen.eval (e.g., event-based metrics include PKBIAS, PKTE and EVBIAS) may be time consuming to compute
metric_exclude		Yes	List of strings	[PKBIAS, PKTE, EVBIAS, FBIAS]	List of metrics to be excluded from metric_subset for calculation (leave blank if no need to exclude any metrics). For example, one could excluded the event-based metrics from being calculated to save compute time
flow_threshold_categorical		Yes	Float	0.9	Non-exceedance probability threshold of streamflow to be used for categorical metrics in ngen.eval. If not set, default to 0.9
flow_threshold_event		Yes	Float	0.9	Non-exceedance probability threshold of streamflow to be used for event-based metrics in ngen.eval. If not set, default to 0.9
lead_times		No	List of strings or integers	[all, 1-5, 6-10, 11-18]	List of lead times (in hours) for which metrics should be calculated for. The lead times can be integers (e.g., 1, 2), or an interval (e.g., 1-3), or 'all'. If set to 'all', all raw lead times (i.e., 1, 2, ..., 18 in the case of short-range forecasts) will be calculated. An interval based lead time (e.g., 1-3 hours) indicates calculating metrics using data pooled from a group of raw lead times (e.g., 1, 2, and 3).

Explanation for section 7: **plots**

The section is used to set up configurations for visualizing metrics and exporting to png files. ngen-verf will generate three different types of plots: spatial maps, histograms, and boxplots.

- 1) For histogram, a plot comparing the datasets is generated for each lead time and each metric
- 2) For boxplot, a plot comparing the datasets and different lead times is generated for each metric
- 3) For spatial map, a plot is generated for each dataset, lead time, and metric

For sample plots in each category, check out [Sample plots](#)

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
histograms	plot	No	Boolean	True	Whether to create histogram plots
	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create histogram plots for. This list must be a subset of the list of metrics calculated, as defined by metrics_subset in metrics section
	binning	Yes	Dictionary	KGE: [-inf, -1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0] NSE: [-inf, -1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0] NNSE: [0, 0.2, 0.4, 0.6, 0.8, 1.0] CORR: [-1, -0.5, 0, 0.2, 0.4, 0.6, 0.8, 1.0]	For each metric, define a list of bin edges for generating the histogram plot For 'KGE','NSE','CORR' and 'NNSE', default bins (defined in settings.py) will be used if not defined. For other metrics, the data will be cut into 8 equal-width bins if bin edges are not provided, which may result in improperly displayed statistics due to outliers.
	lead_times	No	List of integers or strings	[1, 3, 5, 10, 15, 18, 1-5, 6-10, 11-18]	List of lead times (in hours) to create histograms for. Notes: 1) lead times can be either selected raw lead times (e.g., 1, 2, ..., 18 for short-range forecasts) or grouped lead times (e.g., 1-5 hours), where selected raw lead times are grouped together to compute statistics 2) lead times must be a subset of lead times defined for metrics calculation by lead_times in metrics section

	tag	Yes	String	test1	A string that gets added to the filename of histogram plots to differentiate between plots from verification runs with different configurations, e.g., different lead time groups or different binning scenarios, to avoid overwriting the existing plots. Leave it blank if overwriting is desired
boxplots	plot	No	Boolean	True	Whether to create boxplots
	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create boxplots for. This list must be a subset of the list of metrics calculated, as defined by metrics_subset in metrics section
	lead_times	No	List of integers or strings	[1, 3, 5, 10, 15, 18, 1-5, 6-10, 11-18]	Same as for histograms
	scaling	Yes	Dictionary	KGE: [-0.5,1] NSE: [-0.5,1] CORR: [-0.5,1] NNSE: [0, 1]	Define an appropriate range (min/max) for each metric to scale the boxplot. Default ranges will be applied for 'KGE', 'NSE', 'CORR', and 'NNSE' if not specified. For other metrics, if a range is not defined, values will remain unscaled, which may result in improperly displayed statistics due to outliers.
	show_outliers	Yes	Boolean	false	Whether or not to show the outliers in the boxplots
	tag	No	String	test	Same as for histograms
Spatial maps	plot	No	Boolean	True	Whether to create spatial maps
	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create histogram plots for. This list must be a subset of the list of metrics calculated, as defined by metrics_subset in metrics section
	scaling	Yes	Dictionary	KGE: [-0.5,1] NSE: [-0.5,1] CORR: [-0.5,1] NNSE: [0, 1]	Same as for boxplots
	lead_times	No	List of integers or strings	[1, 3, 5, 10, 15, 18, 1-5, 6-10, 11-18]	Same as for histograms
	tag	Yes	String	test1	Same as for histograms

Verification datasets and outputs

Sample screen output

```
INFO:__main__: Config file to use: config_test_conus.yaml
INFO:ngen.verf.identify_location_ids: Total number of locations for dataset v3_may: 8159
INFO:ngen.verf.identify_location_ids: Total number of locations for dataset v3_jul: 8159
INFO:ngen.verf.utils: Using 3 workers, with ~3 GB per worker.
INFO:ngen.verf.fetch_data: ===== Fetch data for NWM dataset v3_may: nwm30 short_range 2024-05-01 to 2024-05-31 =====
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-01 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-02 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-03 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-04 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-05 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-06 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-07 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-08 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-09 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-10 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-11 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-12 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-13 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-14 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-15 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-16 ...
```

```
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-17 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-18 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-19 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-20 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-21 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-22 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-23 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-24 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-25 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-26 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-27 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-28 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-29 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-30 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-05-31 ...
INFO:ngen.verf.fetch_data: NWM forecast data are saved in parquet files at: /home/yuqiong.liu/work/data/ngen-
verf/test_all_conus/nwm30/timeseries/short_range
INFO:ngen.verf.fetch_data: ===== Fetch data for NWM dataset v3_jul: nwm30 short_range 2024-07-01 to 2024-07-
31 =====
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-01 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-02 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-03 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-04 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-05 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-06 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-07 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-08 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-09 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-10 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-11 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-12 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-13 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-14 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-15 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-16 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-17 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-18 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-19 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-20 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-21 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-22 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-23 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-24 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-25 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-26 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-27 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-28 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-29 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-30 ...
INFO:ngen.verf.fetch_data: Retriving NWM forecast data for 2024-07-31 ...
INFO:ngen.verf.fetch_data: NWM forecast data are saved in parquet files at: /home/yuqiong.liu/work/data/ngen-
verf/test_all_conus/nwm30/timeseries/short_range
INFO:__main__: Execution time for fetch_fcst_data: 7693.861318588257 seconds
INFO:ngen.verf.utils: Using 3 workers, with ~3 GB per worker.
INFO:ngen.verf.fetch_data: Downloading USGS data for 2024-05-01 00:00:00 to 2024-05-31 00:00:00 ...
INFO:ngen.verf.fetch_data: Downloading USGS data for 2024-07-01 00:00:00 to 2024-07-31 00:00:00 ...
INFO:ngen.verf.fetch_data: USGS observation data are saved in parquet files at: /home/yuqiong.liu/work/data
/ngen-verf/test_all_conus/usgs
INFO:__main__: Execution time for fetch_obs_data: 561.9178457260132 seconds
INFO:ngen.verf.pair_data: Joining time series for dataset v3_may in DuckDBDatabase: /home/yuqiong.liu/work/data
/ngen-verf/test_all_conus/teehr.db
INFO:ngen.verf.pair_data: Exporting paired data for group 0 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group0.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 1 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group1.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 2 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group2.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 3 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group3.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 4 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group4.parquet ...
```

```

INFO:ngen.verf.pair_data: Exporting paired data for group 5 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group5.parquet ...
...
INFO:ngen.verf.pair_data: Exporting paired data for group 33 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group33.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 34 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group34.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 35 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group35.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 36 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group36.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 37 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_may.joined.group37.parquet ...
INFO:ngen.verf.pair_data: Joining time series for dataset v3_jul in DuckDBDatabase: /home/yuqiong.liu/work/data
/ngen-verf/test_all_conus/teehr.db
INFO:ngen.verf.pair_data: Exporting paired data for group 0 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group0.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 1 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group1.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 2 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group2.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 3 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group3.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 4 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group4.parquet ...
...
INFO:ngen.verf.pair_data: Exporting paired data for group 35 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group35.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 36 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group36.parquet ...
INFO:ngen.verf.pair_data: Exporting paired data for group 37 locations to /home/yuqiong.liu/work/data/ngen-verf
/test_all_conus/joined/v3_jul.joined.group37.parquet ...
INFO:__main__: Execution time for pair_data: 9015.612207174301 seconds
INFO:ngen.verf.calc_metrics: Metrics will be calculated using ngen.eval library
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 0 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 1 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 2 ...
...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 33 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 34 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 35 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 36 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_may group 37 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 0 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 1 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 2 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 3 ...
...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 33 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 34 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 35 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 36 ...
INFO:ngen.verf.calc_metrics: Calculating metrics for v3_jul group 37 ...
INFO:__main__: Execution time for compute_metrics: 2570.8697843551636 seconds
INFO:ngen.verf.create_plots: Spatial maps created at: /home/yuqiong.liu/work/data/ngen-verf/test_all_conus
/plots/maps
INFO:ngen.verf.create_plots: Histograms created at: /home/yuqiong.liu/work/data/ngen-verf/test_all_conus/plots
/histograms
INFO:ngen.verf.create_plots: Boxplots created at: /home/yuqiong.liu/work/data/ngen-verf/test_all_conus/plots
/boxplots
INFO:__main__: Execution time for plot_metrics: 55.1886248588562 seconds

```

Directory structure

```
-- calib_basin_group1
| -- joined
|   | -- v3_oct.joined.group0.parquet
|   `-- v3_sep.joined.group0.parquet
| -- metrics
|   | -- v3_oct.metrics.parquet
|   `-- v3_sep.metrics.parquet
| -- nwm30
|   | -- timeseries
|   `-- zarr
| -- plots
|   | -- boxplots
|   | -- histograms
|   `-- maps
| -- usgs
|   | -- 2024-09-01.parquet
|   | -- 2024-09-02.parquet
|   | -- 2024-09-03.parquet
|   | -- 2024-09-04.parquet
|   | -- 2024-09-05.parquet
```

```

|   | -- 2024-10-25.parquet
|   | -- 2024-10-26.parquet
|   | -- 2024-10-27.parquet
|   | -- 2024-10-28.parquet
|   | -- 2024-10-29.parquet
|   | -- 2024-10-30.parquet
|   `-- 2024-10-31.parquet
| -- v3_oct
|   `-- fcst
|-- v3_sep
|   `-- fcst
```

16 directories, 65 files

Notes on the Directory Structure

1. **calib_basin_group1** corresponds to the `dataset_name` field specified in the YAML configuration. It serves as the root directory, storing all datasets and outputs generated during the verification process.
2. **Step 1: Fetch NWM Forecast Data (fetch_fcst_data)**
 - Downloads and processes NWM forecast data for the specified locations and time periods.
 - Stores raw forecast data in the `nwm30/zarr` subdirectory.
 - Retrieve the data for the selected location and saves it in `nwm30/timeseries` in Parquet format.
 - Creates symbolic links to the required Parquet files for each dataset (e.g., `v3_oct/fcst`, `v3_sep/fcst`).
3. **Step 2: Fetch Streamflow Observation Data (fetch_obs_data)**
 - Downloads USGS streamflow data.
 - Stores observations as Parquet files (by day as configured) in the `usgs` subdirectory.
4. **Step 3: Pair Forecast and Observation Data (pair_data)**
 - Generates paired datasets and saves them as Parquet files in the `joined` subdirectory for each location group (e.g., `v3_oct_joined.group0.parquet`, `v3_oct_joined.group0.parquet`).
 - The number of location groups is calculated based on the `group_size` defined in the [pair_data](#) section, and the total number of locations

5. Step 4: Compute Metrics (`compute_metrics`)

- Generates a metrics Parquet file for each dataset (combining all location groups).

6. Step 5: Plot Metrics (`plot_metrics`)

- Creates three types of plots and saves them as PNG files in the following subdirectories:
 - **Boxplots:** plots/boxplots
 - **Histograms:** plots/histograms
 - **Maps:** plots/maps

Notes on NWM Forecast Data

- **Raw NWM Forecast Data**

The downloaded NWM forecast data is stored in JSON format at:

`[data_dir_root]/[location_set_name]/[nwm_version]/zarr/[nwm_configuration]`,

with one file per lead time per forecast hour. Example:

```
/home/yuqiong.liu/work/data/ngen-verf/calib_basin_group1/nwm30/zarr/short_range/nwm.20241031.nwm.t23z.short_range.channel_rt.f018.conus.nc.json
```

- **Processed NWM Forecasts**

Forecast data for locations specified in either `location_list` or `location_list_file` is processed and stored in Parquet format at:

`[data_dir_root]/[location_set_name]/[nwm_version]/timeseries/[nwm_configuration]`

Example:

```
/home/yuqiong.liu/work/data/ngen-verf/calib_basin_group1/nwm30/timeseries/short_range/20241031T23.parquet
```

- **Dataset-Specific Storage and Symbolic Links**

A subdirectory is created for each dataset under:

`[data_dir_root]/[location_set_name]/[dataset_name]`

Within this directory, symbolic links to the relevant Parquet files are generated. This allows datasets to selectively access only the required NWM forecasts, based on parameters such as `nwm_version`, `nwm_configuration`, `forecast_start_time`, and `forecast_end_time` specified in the dataset's configuration.

- **Efficient Data Handling**

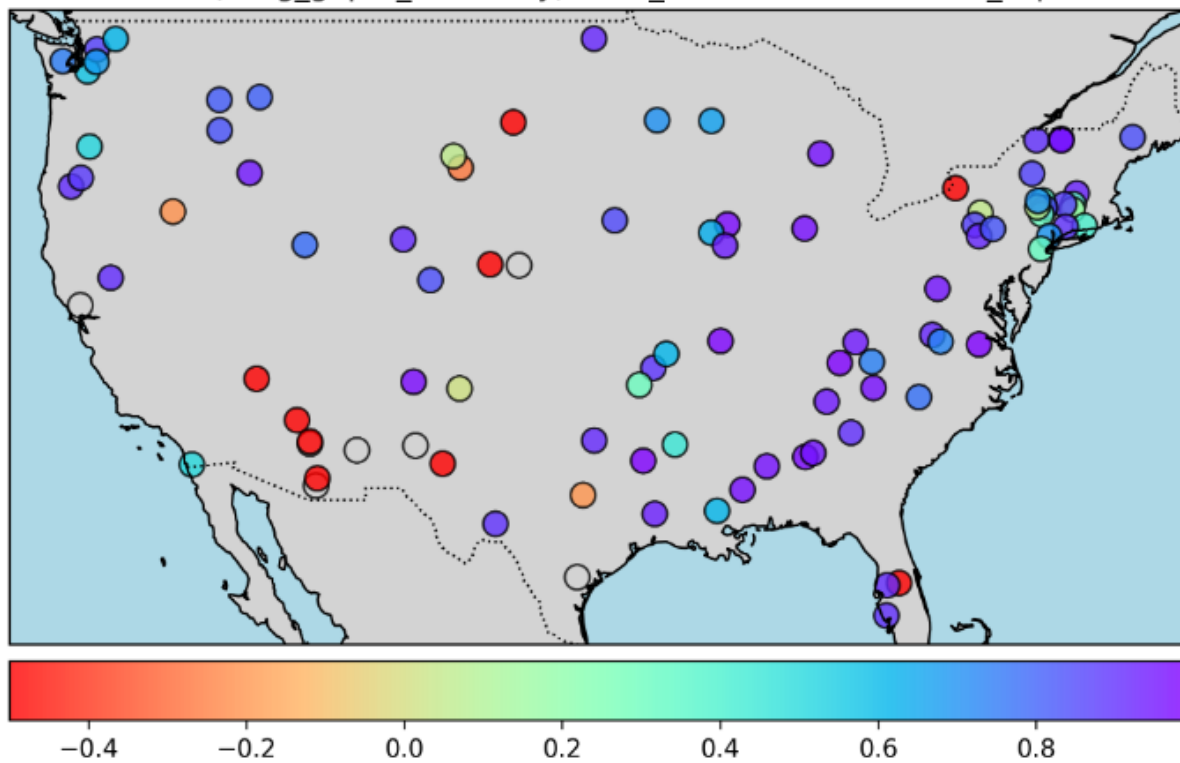
The symbolic link structure enables flexible dataset adjustments without requiring re-downloading or reprocessing of NWM forecast data, which can be resource-intensive. The subsequent **data pairing step (`pair_data`)** is conducted using these symbolic links, ensuring efficiency and minimizing redundant computations.

Sample plots

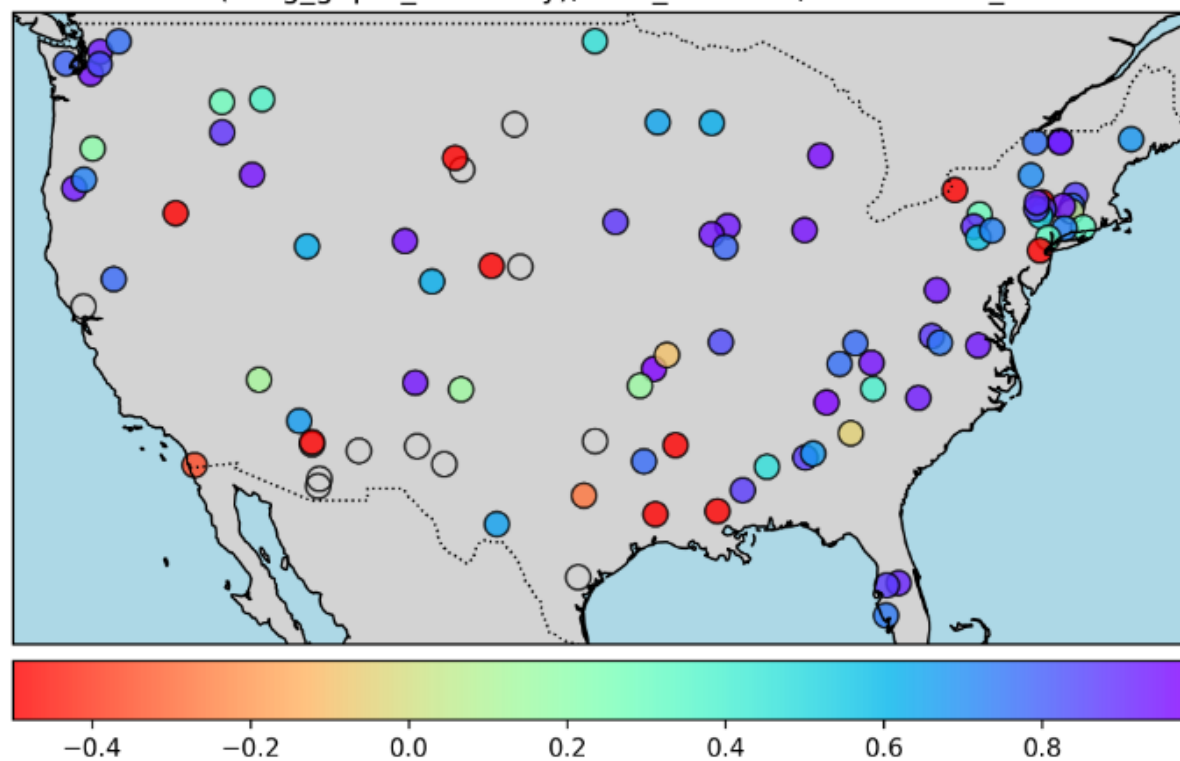
Sample spatial maps

Spatial maps of KGE from the first lead hour comparing the two datasets (left PNG: `v3_sep`, right PNG: `v3_oct`). Open circles indicate NA values (i.e., the metric could not be calculated)

KGE (kl_{ing}_g_up_ta_efficiency), lead_time=1h, dataset=v3_sep

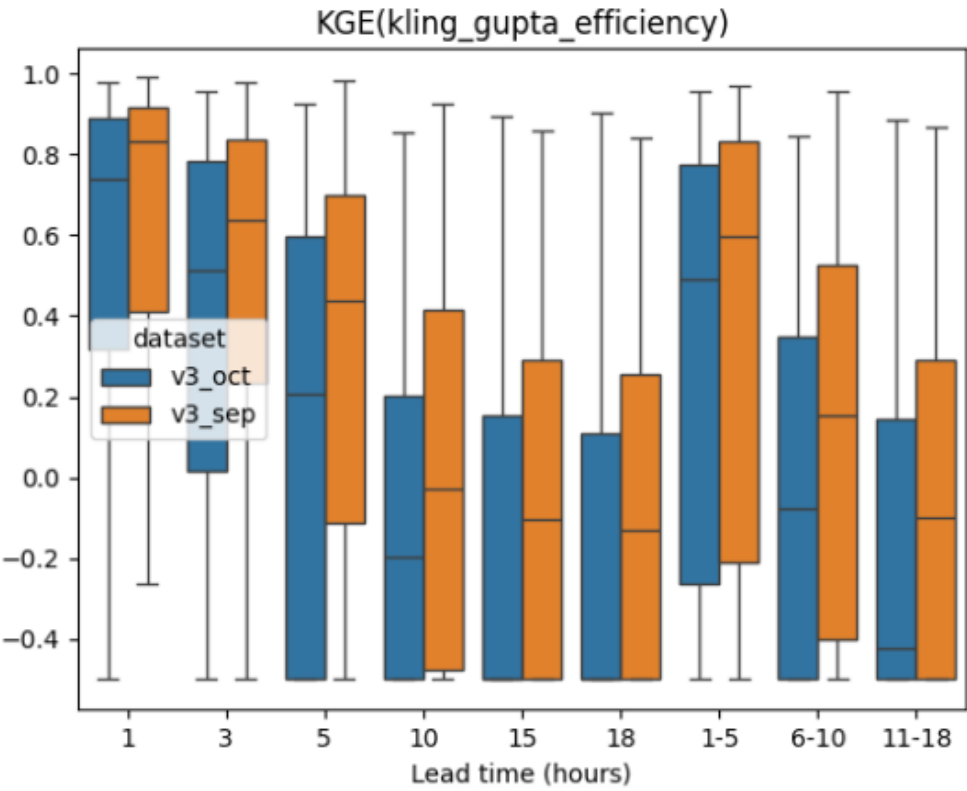


KGE (kl_{ing}_g_up_ta_efficiency), lead_time=1h, dataset=v3_oct



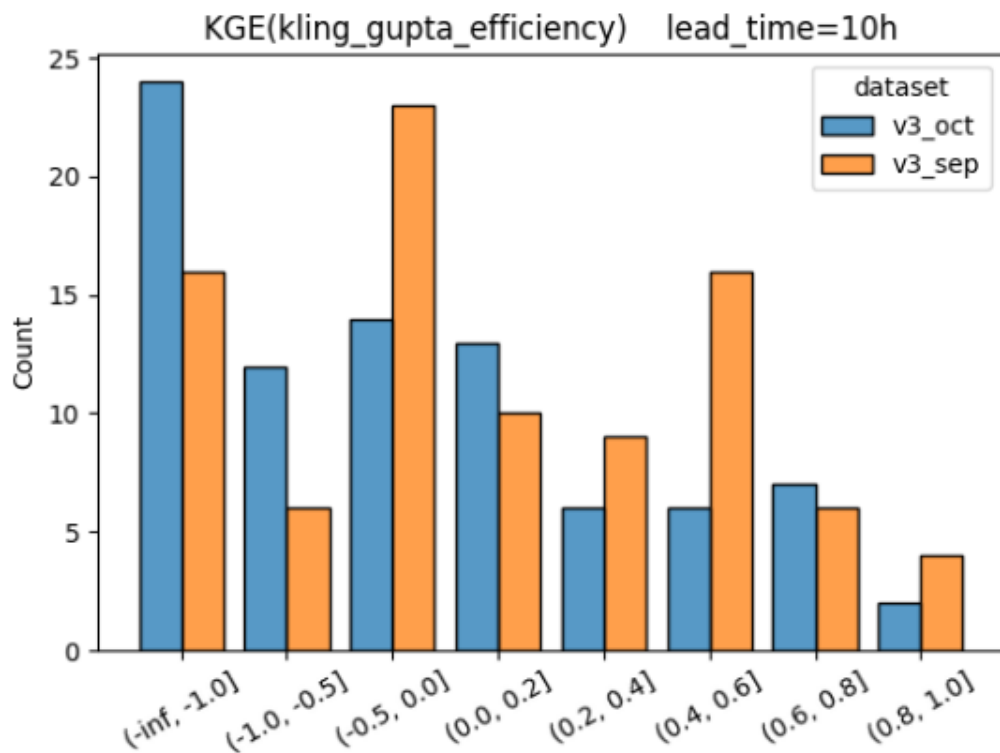
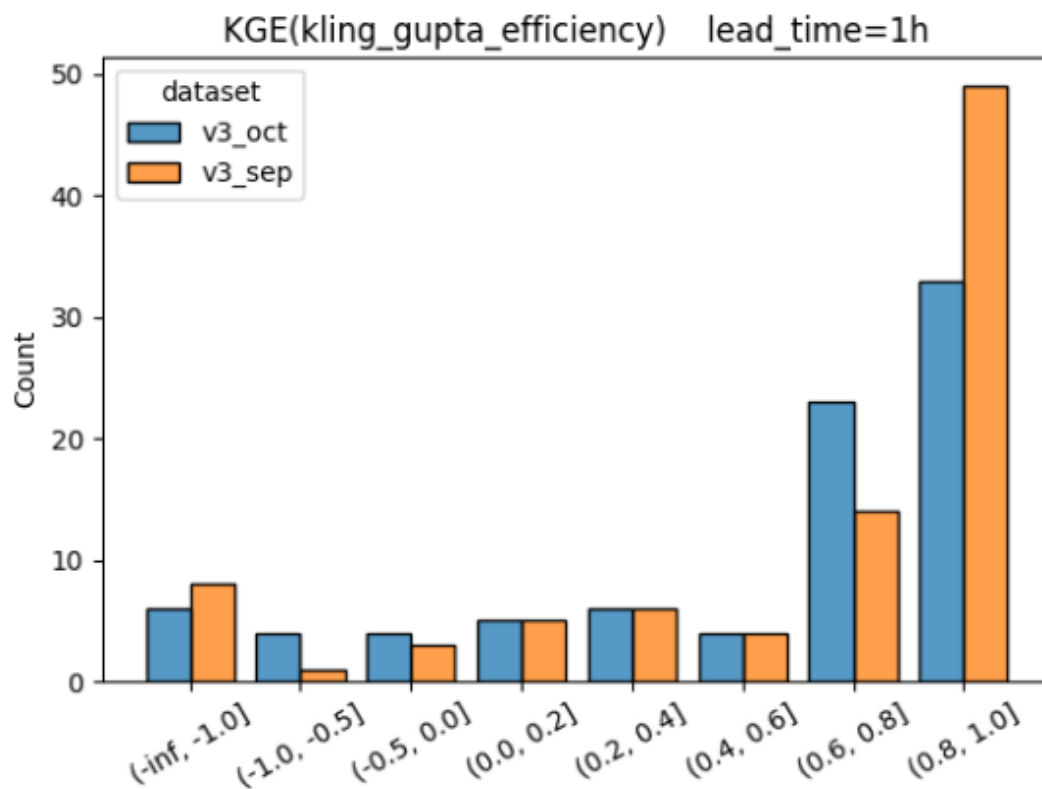
Sample boxplots

Boxplot summarizing KGE from all locations for selected lead times as defined in the configuration, comparing the two datasets (v3_sep vs v3_oct) side by side in the same figure.



Sample histograms

Histograms summarizing KGE from all locations for the two datasets (left PNG: lead_time=1h, right PNG: lead_time=10h)



Metrics Supported

ngen-verf currently supports metric calculation using either the TEEHR library or the ngen-eval library. The metrics supported by each library are listed below.

Metric Short Name	Metric Long Name	Supported by TEEHR?	Supported by ngen-eval?
CORR	Pearson correlation	Yes	Yes
sCORR	Spearman correlation	Yes	No
NSE	Nash Sutcliffe Efficiency	Yes	Yes
NNSE	Normalized Nash Sutcliffe Efficiency	Yes	Yes
NSElog	logrithmic Nash Sutcliffe Efficiency	No	Yes
NSEwt	weighted NSE and NSElog	No	Yes
KGE	Kling Gupta Efficiency	Yes	Yes
KGE1	Kling Gupta Efficiency mod1	Yes	No
KGE2	Kling Gupta Efficiency mod2	Yes	No
ME	mean error	Yes	No
MAE	mean absolute error	Yes	Yes
MSE	mean squared error	Yes	No
RMSE	root mean squared error	Yes	Yes
RMAE	mean absolute relative error	Yes	No
PBIAS	percent bias	No	Yes
RBIAS	relative bias	Yes	No
MBAIS	multiplicative bias	Yes	No
aprBIAS	annual peak relative bias	Yes	No
R2	r squared	Yes	No
RSR	RMSE observation standard deviation ratio	No	Yes
HSEG_FDC	percent bias in high flow section of flow duration curve	No	Yes
MSEG_FDC	percent bias in medium flow section of flow duration curve	No	Yes
LSEG_FDC	percent bias in low flow section of flow duration curve	No	Yes
POD	probability of detection	No	Yes
FAR	false alarm ratio	No	Yes
CSI	critical success index	No	Yes
FBIAS	frequency bias	No	Yes
PKBIAS	percent peak flow bias	No	Yes
PKTE	peak timing error	No	Yes
EVBIAS	event volume bias	No	Yes
n_obs	observation count	Yes	No
n_mod	simulation count	Yes	No
min_obs	observation minimum	Yes	No
min_mod	simulation minimum	Yes	No
max_obs	observation maximum	Yes	No
max_mod	simulation maximum	Yes	No
mean_obs	observation average	Yes	No
mean_mod	simulation average	Yes	No
sum_obs	observation sum	Yes	No
sum_mod	simulation sum	Yes	No
var_obs	observation variance	Yes	No
var_mod	simulaiton variance	Yes	No
max_delta	max value delta	Yes	No
pt_obs	observation maximum value time	Yes	No
pt_mod	simulation maximum value time	Yes	No

pt_err	maximum value time delta	Yes	No
--------	--------------------------	-----	----